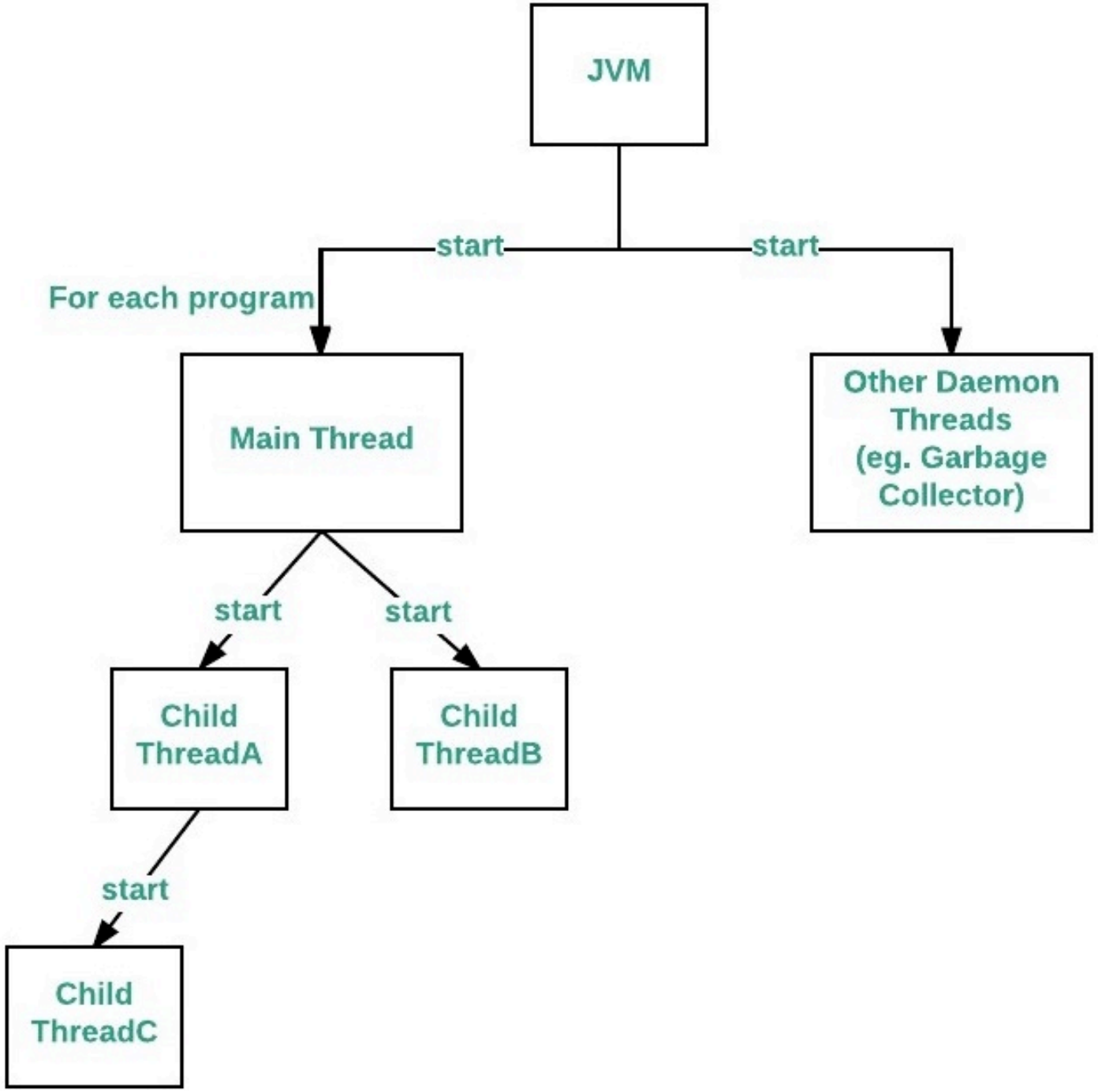
Konkurentno programiranje

Stefan Antić

santic@raf.rs


JVM
start
start
For each program
Main Thread
Other Daemon Threads (eg. Garbage Collector)
start
start
Child ThreadA
Child ThreadB
start
Child ThreadC

Uvod

- Java sadrži biblioteke potrebne za konkurentno programiranje
- Izvršavanje konkurentnih programa zavisi od Java virtuelne mašine, OS-a i hardvera
- Deo Java virtuelne mašine koji određuje koji tred će se u kom trenutku izvršavati je **Thread scheduler**

Process

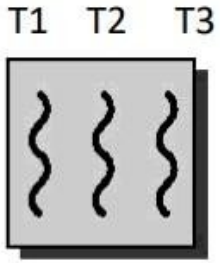
- Proces je bilo koji program koji se trenutno izvršava
- (notepad, calculator...)
- Može da kreira nove procese, **Child Procese**
- Koristi sopstvene resurse, nezavisno od drugih procesa

Thread (nit)

- Nit predstavlja tok izvršavanja. Jedan proces (pokrenut program) može da ima više aktivnih niti.
- Sve niti unutar procesa dele memoriju tog procesa, tj. mogu da pristupaju istim promenljivama (objektima u Java).
- Programer ne može da kontroliše raspoređivanje niti - ono se dešava pri runtime-u.

Thread

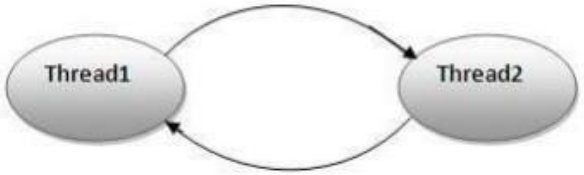
one process
one thread



one process
multiple threads

Create: t1 = new Thread(s1)
Start: t1.start()
Join/wait: t1.join()

Sleep(), yield()



Threads

- Pisanje programskih niti u Javi
  - Nasleđivanje klase Thread
  - Implementacija interfejsa Runnable
  - Implementacija interfejsa Callable<>
  - Anonimna implementacija interfejsa Runnable

Thread states

- Nit se može naći u jednom od šest stanja:
  - new (nakon instanciranja),
  - runnable (nakon poziva start metode),
  - blocked (privremeno neaktivna),
    - kada nit pokuša da pristupi nekom resursu kog je zaključala neka druga nit.
  - waiting (privremeno neaktivna),
    - kada nit čeka da druga nit obavesti raspoređivač o nekom uslovu.
  - timed waiting (isto kao i waiting, uz timeout),
  - terminated (metod run se završio prirodno, ili zbog neuhvaćenog izuzetka)

Threads

- Nova nit se pravi pozivanjem thread.start() metode.
- Start() interno pravi novu nit koja ce izvršiti kod iz run() metode
- Pozivanje thread.run() direktno, ne pravi novu nit, već će se metoda run() izvršiti na Main Thread-u.

Blokirajuće metode Thread klase

- **JOIN** – kada želimo da sačekamo da nit koju smo startovali dođe do svog prirodnog završetka.

```
t1.start(); //startuje nit t1, i ona se izvršava konkurentno sa trenutnom
t1.join(); //blokira trenutnu nit sve dok se t1 ne završi
```

- **SLEEP** – pauziranje izvršavanja trenutne niti.

Data Race ili stanje trke

- Situacija u kojoj dve niti konkurentno (istovremeno) menjaju vrednost na istoj lokaciji u memoriji. Posledica stanja trke će biti da se jedna od tih modifikacija izgubi.Ne možemo garantovati rezultat ovakve operacije.
- Rezultat zavisi od Thread Schedulera.

x++;	x--;
- - - - - - - - -	- - - - - - - - -
R0 ← x	R1 ← x
inc R0	dec R1
R0 → x	R1 → x

Locks

- Korišćenjem mehanizma zaključavanja objekta osigurava se da najviše jedna nit može da pristupi zaključanom objektu u jednom trenutku.
- U Javi ovaj mehanizam se implementira pomoću:
  - **Synchronized bloka,**
  - **Synchronized metode.**

Locks

- Svaki synchronized block je sinhronizovan na jednom objektu.
- Nit može pristupiti samo jednom synchronized block-u u istom objektu u jednom trenutku.

Locks

```
incr(A){
   A.count = A.count + 1;
}
```

=> DATA RACE

```
incr(A){
   synchronized(A){  // acquired A
      A.count = A.count + 1;
   }  // release A
}
```

Locks

```
X{buf, in, out} // neki buffer
Insert(item){
   buf[id] = item;
   id++;
}
Remove(){
   item = buf[out];
   out--;
   return item;
}
```

```
X{buf, in, out} // neki buffer
Insert(item){
   synchronized(X){
      buf[id++] = item;
   }
}
Remove(){
   synchronized(X){
      item = buf[out--];
   }
   return item;
}
```

wait if X is full

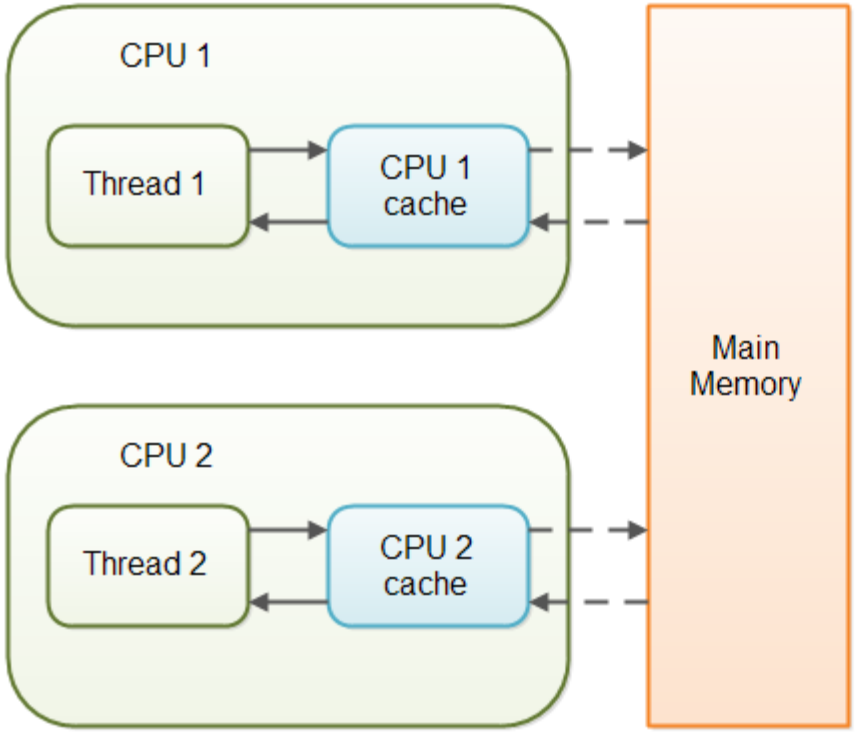
Notify all

wait if X is empty

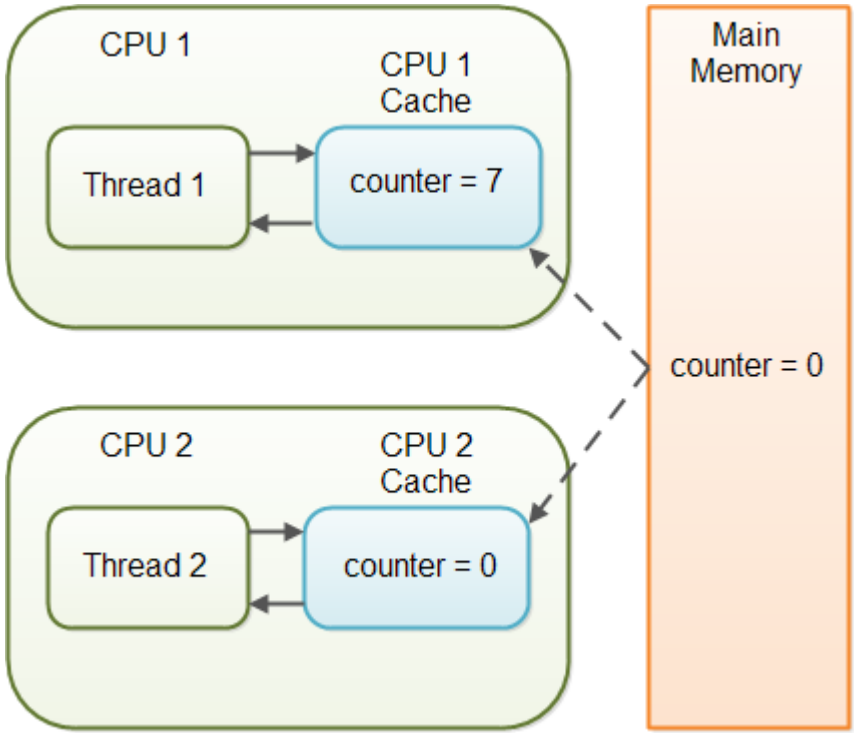
Notify all

Volatile

- Promenljiva označena sa **volatile** govori procesoru da nikada ne kešira vrednost te promenljive



Volatile


CPU 1
CPU 1
Cache
Thread 1
counter = 7
Main
Memory
counter = 0
CPU 2
CPU 2
Cache
Thread 2
counter = 0

Atomic Variables

- Služe da izvrše atomičke operacije (sabiranje, oduzimanje...)
- Imaju get i set metode, koje se ponašaju kao da čitamo/ upisujemo vrednost u volatile promenljivu.
- Nije potrebna sinhronizacija.

Atomic Variables

```
class AtomicCounter {
   private AtomicInteger a = new AtomicInteger(0);

   public void increment() {
      a.incrementAndGet();
   }

   public void decrement() {
      a.decrementAndGet();
   }

   public int value() {
      return c.get();
   }
}
```

- Postoji metoda
  - compareAndSetcompareAndSet(expect, update)

Semaphore

- Najčešće se koriste da ograniče broj niti koje mogu da pristupe nekom resursu
- Semafor sadrži broj dozvola (prvi argument konstruktora) koje se uzimaju metodom acquire, a vraćaju metodom release
- Drugim parameterom konstruktora se omogućava fer redosled (true – fer, false – ne garantuju da ce biti fer).
- Ukoliko je broj dozvola jednak nuli prilikom poziva metode acquire nit će da sačeka da neka druga nit oslobodi jednu dozvolu.

THREAD POOLS

- **Executors.newCachedThreadPool()** - Thread pool koji kreira nove thread-ove po potrebi, ali ce koristiti prethodno kreirane thread-ove kada je to moguce.

- **Executors.newFixedThreadPool(5)** - Thread pool sa fiksnim brojem thread-ova u njemu.

- **Executors.newSingleThreadExecutor() -** Thread pool sa samo jednim thread-om.

- **Executors.newScheduledThreadPool(5)** - Thread pool u kom mozete da birate kada ce se neki thread izvrsiti
  - executorServiceScheduled.schedule(new Runnable(), 3000, TimeUnit.MILLISECONDS)

CyclicBarrier

- Barijera zaustavlja sve niti dok se interni brojač ne dovede na nulu tada se sve niti deblokiraju
- Barijera blokira nit koja je pozvala metodu await i smanjuje interni brojač, kada brojač dospe do nule, sve niti se deblokiraju

CountDownLatch

- Blokira nit dok se interni brojač ne dovede na nulu
- Nit koja pozove await metodu se blokira
- Svaki poziv countDown metode smanjuje interni Brojač kada brojač dođe do nule, blokirana nit se deblokira
- Služi da jedna nit sačeka da ostale niti obave neki posao, tek kada sve niti obave posao, ova nit se deblokira